

CareerCompass AI

Project Concept Note

Submitted by: Hrishikesh Dutta, Arnab Kumar Kashyap, Kasturi Kashyap and Shashwati Shivam

1. Project Title and Application Name

Project Title: AI-Powered Career Recommendation Engine

Application Name:

CareerCompass AI -

CareerCompass AI is a full-stack, AI-powered career guidance web application that converts a structured user profile into personalised, ranked career path recommendations, built end-to-end using AI-assisted (“Vibe Coding”) development practices.

2. Problem Statement / Objective

Career decision-making is often approached through unstructured, generic advice that does not account for an individual’s actual education, experience, skills, and working preferences. Open-ended AI chat tools can offer suggestions, but their output is inconsistent in format, difficult to compare across options, and rarely broken down into an actionable next step.

Objective: To design and deploy a functional, publicly accessible web application that collects a structured user profile through a guided wizard, sends it to a Large Language Model via a secure backend, and returns a consistent, comparable, and actionable set of career path recommendations — rendered progressively in the UI as the model streams its response.

3. Target User and Use Case

- **Primary users:** Students and early-career professionals evaluating potential career directions based on their education, skills, and interests.
- **Secondary users:** Career changers seeking a structured, repeatable way to compare paths against their existing skill set and risk tolerance.
- **Core use case:** A user completes a six-step guided questionnaire (education level, experience range, self-reported skills, domain interests, preferred work style, and risk tolerance). The application returns three to five ranked career paths, each with a calibrated match score, a rationale tied to the user’s specific inputs, a concrete skills-gap list, and one actionable first step.

4. LLM Model and API Used

Component	Details
LLM Provider	Google Gemini API
Model(s)	gemini-2.5-flash (primary), with automatic fallback to gemini-1.5-flash
Integration Pattern	Backend-only integration via <code>generate_content_stream</code> ; API key is never exposed to the frontend
Reliability Handling	Multi-model retry loop with exponential backoff on HTTP 503/429 errors
Response Delivery	Server-Sent Events (SSE) streamed from FastAPI to the browser over <code>POST /api/recommend</code>

5. Key Features of the Application

- **Structured 6-step wizard** — validated, typed inputs (education, experience, skills, interests, work style, risk tolerance) rather than a free-text prompt box.
- **Streaming, progressive UI** — SSE-based streaming with mid-stream partial-JSON parsing, so recommendation cards appear one-by-one as they resolve rather than after a full-response delay.
- **Calibrated match scoring** — an enforced scoring rubric (60–98) baked into the system prompt, producing consistent, comparable output across users.
- **Bento-grid results layout** — each recommendation rendered as a structured card with rationale, skills-gap list, and a single actionable first step.
- **Shimmer skeleton loading state** — animated placeholder cards fill unresolved slots and are progressively replaced by real content as the stream resolves.
- **Mock mode fallback** — the application remains demonstrable even without a live API key, using a mock response built from the user’s actual inputs.
- **Secure-by-design backend** — the frontend calls only its own backend endpoint; the LLM API key lives exclusively in server-side environment variables.
- **Production-grade deployment** — multi-stage Docker build, Terraform-provisioned AWS infrastructure (ECR + App Runner), and an AWS CodeBuild CI pipeline.

6. Expected User Experience and Outcomes

- **Guided, low-friction input:** the six-step wizard keeps each question focused and validated, with a visible progress bar and disabled navigation until each step is answered.
- **Perceived responsiveness:** progressive card streaming removes the 10–15 second “blank spinner” delay typical of full-response AI tools, replacing it with an incrementally-filling results grid.
- **Actionable, comparable output:** users leave with a ranked shortlist of career paths, each with a clear rationale, a specific skills gap, and one concrete next action — rather than an unstructured block of advice.
- **Cross-device usability:** a responsive bento-grid layout (3 columns on desktop, collapsing to 2 and then 1 on smaller viewports) ensures the experience is functional on both desktop and

mobile browsers.

- **Reliability:** multi-model fallback and mock mode ensure the application degrades gracefully rather than failing outright if the primary model or API key is unavailable.

7. Live Application

Live AWS Application URL: <https://dckwuhsairu3m.cloudfront.net>

The application is deployed on AWS and publicly accessible via HTTPS through an AWS CloudFront distribution in front of an AWS App Runner service running the containerised FastAPI backend (which also serves the built React frontend as static files). The application is fully functional at the above URL at the time of submission.

Property	Value
Live URL	https://dckwuhsairu3m.cloudfront.net
Protocol	HTTPS (via AWS CloudFront)
Compute	AWS App Runner (containerised FastAPI backend)
Container Registry	Amazon ECR (eu-north-1)
Infrastructure as Code	Terraform (provisions ECR + App Runner)
CI Pipeline	AWS CodeBuild (buildspec.yml)

GitHub Repository: <https://github.com/HyperPenetrator/CareerAI>